

Arquitetura de Sistemas Nativos de Inteligência Artificial: O Paradigma do Utilizador-Agente e Identidade Federada na Cloudflare

1. Fundamentos e Paradigma Arquitetural do Utilizador-Agente

A transição de arquiteturas de software baseadas em servidores sem estado (stateless) para sistemas nativos de Inteligência Artificial (IA) exige uma reavaliação profunda da forma como a identidade, a memória e a computação são geridas. Em sistemas tradicionais de Software as a Service (SaaS), os dados dos utilizadores são frequentemente agrupados em bases de dados monolíticas relacionais ou não relacionais, com chaves de encriptação geridas centralmente. Este modelo clássico introduz latência, complexidade na separação de inquilinos (multi-tenancy) e, mais criticamente, um vetor de ataque unificado onde o comprometimento do servidor expõe os dados de todos os utilizadores em texto simples.

A proposta arquitetural aqui detalhada estabelece um sistema modular e orientado a IA onde cada utilizador registado não é apenas uma entrada passiva numa tabela de base de dados, mas sim um "agente" computacional vivo, isolado e autossuficiente, concebido para operar como um sistema de identidade universal, comparável a um "Apple ID" descentralizado. Sob este modelo, a infraestrutura global da Cloudflare é utilizada para instanciar um "Twin-User" (Utilizador-Gémeo) para cada identidade registada. Este agente retém a sua própria memória, incluindo registos de navegação, histórico transacional e contexto semântico, gere as suas chaves de autenticação e criptografia de dados sensíveis de forma estritamente isolada, e atua como uma âncora de identidade segura para transitar entre múltiplos sistemas e plataformas. O objetivo central desta arquitetura é construir um banco de dados de identidades ativas. Cada vez que um utilizador se regista, não está apenas a criar credenciais, mas a instanciar um ambiente computacional dedicado (um *Durable Object*) que o representará digitalmente. Este ambiente permite que o utilizador conceda acesso aos seus dados a outros sistemas de forma anonimizada, garantindo que as chaves de encriptação primárias nunca são expostas publicamente e que o fornecedor da infraestrutura (Cloudflare) atua sob uma premissa de conhecimento zero (Zero-Knowledge).

2. A Fundação da Identidade: Registo, Acesso e Trânsito Multi-Sistema

Para responder ao desafio de como registar um utilizador novo e como permitir que este banco de dados de identidades possa ser usado em outras situações, a arquitetura afasta-se dos repositórios centralizados de palavras-passe. O registo e o acesso são redefinidos através da convergência de protocolos criptográficos modernos suportados diretamente pelos

navegadores web e sistemas operativos, criando uma identidade portátil e altamente segura. A espinha dorsal deste processo de registo baseia-se na especificação WebAuthn e no uso de *passkeys*. Quando um utilizador pretende registar-se no sistema base, o seu dispositivo local (seja um smartphone com reconhecimento facial ou um computador portátil com leitor de impressões digitais) gera um par de chaves criptográficas de forma nativa e isolada no seu hardware seguro (Secure Enclave). O servidor central da Cloudflare, através da sua camada de proteção de acesso, recebe apenas a chave pública. Este mecanismo erradica a vulnerabilidade inerente às palavras-passe tradicionais e impede tentativas de roubo através de engenharia social ou *phishing*. O ato de registo cria a âncora da identidade, que será simultaneamente utilizada para derivar as chaves de encriptação dos dados da memória do agente, garantindo que apenas o portador do dispositivo original possua o material necessário para desbloquear os seus dados.

2.1. O Utilizador como Provedor de Identidade (O Modelo "Apple ID")

Uma vez registada, esta identidade necessita de ser acedida e transitada por diversos sistemas ou plataformas de terceiros que o utilizador deseje utilizar, funcionando como uma identidade universal. Para alcançar este comportamento, cada "Twin-User" atua tecnicamente como o seu próprio Provedor de Identidade (Identity Provider - IdP) através da implementação do protocolo OpenID Connect (OIDC).

Em vez de depender de um serviço central monolítico para autenticar o utilizador em sistemas externos, o Worker associado ao agente do utilizador implementa a biblioteca `@cloudflare/workers-oauth-provider`. Esta biblioteca dota o agente da capacidade de atuar como um servidor OAuth 2.1 completo, suportando fluxos como o *Proof Key for Code Exchange* (PKCE), que é imperativo para a segurança em arquiteturas modernas. Quando o utilizador decide fazer *login* num sistema SaaS de terceiros utilizando a sua nova identidade, o sistema de terceiros inicia um fluxo de autorização direcionado ao endpoint OIDC do agente do utilizador. O agente autentica o utilizador localmente (através da validação do *passkey* via WebAuthn) e emite um *JSON Web Token* (JWT) assinado digitalmente.

Este modelo apresenta uma inversão de controlo profunda: o utilizador não cede a sua identidade ao sistema de terceiros; em vez disso, o seu agente pessoal gera um passaporte criptográfico temporário e escopado, atestando a sua autenticidade. Isto permite que a identidade navegue por um número ilimitado de ecossistemas sem que os dados biográficos subjacentes sejam replicados de forma descontrolada.

2.2. Anonimização em Outros Sistemas via Identificadores "Pairwise"

Um dos requisitos mais complexos de um sistema de identidade portátil é impedir que as diversas plataformas onde o utilizador faz *login* consigam conspirar para construir um perfil global (tracking) cruzando identificadores. Se a identidade fornecesse o mesmo identificador global (por exemplo, um ID numérico estático ou um endereço de e-mail) a todos os sistemas, o anonimato seria impossível.

Para assegurar que todos os dados relacionados ao utilizador estejam anonimizados em outros lugares, o protocolo OIDC do agente é configurado para utilizar identificadores de sujeito pseudónimos, conhecidos como Identificadores *Pairwise* (Pairwise Subject Identifiers - PPID). O identificador *Pairwise* é desenhado de forma a que, para um determinado utilizador final, o valor da reivindicação sub (sujeito) no *ID Token* seja um identificador diferente, mas estável, calculado especificamente para cada entidade dependente (Relying Party).

O mecanismo matemático por detrás desta anonimização envolve a aplicação de encriptação determinística avançada, frequentemente utilizando AES no modo SIV (Synthetic Initialization Vector). O agente do utilizador calcula o identificador combinando o ID local e verdadeiro do utilizador com o identificador do setor (Sector ID, tipicamente o domínio do sistema de terceiros) e um *salt* criptográfico local. O resultado é uma cadeia de caracteres opaca. Quando a mesma identidade acede a um sistema de saúde, este recebe o identificador sub=naeta7jee4eTu5a, mas quando acede a uma plataforma financeira, recebe sub=leToonahquiiCai. Ambos os sistemas podem gerir a sessão e a memória do utilizador de forma consistente dentro das suas fronteiras, mas é-lhes matematicamente impossível correlacionar que se trata da mesma pessoa, garantindo a privacidade e a conformidade com normas rigorosas de desvinculação (unlinkability).

Modelo de Identificador	Mecanismo de Geração	Risco de Correlação (Tracking)	Caso de Uso Ideal
Identificador Público	ID global estático (ex: user_123 ou email@dominio.com) emitido para todos os clientes.	Elevado. Plataformas distintas podem cruzar dados e construir perfis paralelos.	Ambientes internos e estritamente controlados onde o rastreio inter-serviços é desejado.
Identificador Pairwise	SIV_{AES}(Sector_ID \parallel Local_Sub \parallel Salt) - Hash irreversível gerado por domínio.	Nulo. O identificador é único por entidade dependente, impossibilitando a desanonimização.	Autenticação em SaaS externos, garantindo privacidade e anonimização total da identidade.

3. Criptografia de Conhecimento Zero e Proteção de Dados Sensíveis

A premissa central de que a chave de criptografia de dados sensíveis nunca pode ser exposta publicamente e que a infraestrutura deve ser concebida como um ambiente *Zero-Knowledge* (Conhecimento Zero) requer a adoção rigorosa de criptografia assimétrica e simétrica do lado do cliente (Client-Side Encryption). Num sistema de utilizadores-agentes onde ficheiros, memória e registos são alojados na nuvem, o provedor da infraestrutura (Cloudflare) armazena os dados, mas nunca detém as chaves para os decifrar.

3.1. Derivação de Chaves através da Extensão WebAuthn PRF

Para que o utilizador não necessite de memorizar e inserir manualmente frases de recuperação complexas ou senhas mestras vulneráveis, o sistema tira partido da Web Crypto API do navegador combinada com a extensão Pseudo-Random Function (PRF) da especificação WebAuthn. Tradicionalmente, o WebAuthn apenas fornece assinaturas digitais, o que é suficiente para autenticação, mas insuficiente para a encriptação direta de dados, uma vez que não expõe a chave privada do hardware.

A extensão PRF resolve esta limitação ao permitir que o cliente combine uma semente (denominada *salt*) fornecida pela aplicação web com um valor privado específico da credencial armazenado no hardware de segurança do dispositivo. Durante o processo de registo ou acesso à identidade, a chamada ao WebAuthn inclui a instrução para avaliar a PRF. O autenticador processa o *salt* e devolve um resultado pseudo-aleatório de 32 bytes de forma

determinística. Este resultado nunca sai do dispositivo do utilizador e não pode ser forjado por um atacante que não possua a posse física e o desbloqueio biométrico do autenticador. Este valor de 32 bytes atua como o material criptográfico base. A aplicação front-end, operando dentro do browser do utilizador, importa este valor para a Web Crypto API utilizando o algoritmo AES-256-GCM. O AES-GCM (Advanced Encryption Standard com Galois/Counter Mode) fornece tanto confidencialidade como integridade autenticada, assegurando que os dados não só estão encriptados, mas que qualquer tentativa de adulteração do texto cifrado será detetada imediatamente no momento da descriptação. Toda a memória do agente, chaves de autenticação de terceiros e registos de navegação são encriptados localmente e submetidos ao servidor central apenas como bolhas de dados ininteligíveis (ciphertext).

3.2. Hybrid Public Key Encryption (HPKE) para Interações e Espelhamento

Enquanto a encriptação simétrica resolve o armazenamento de dados submetidos diretamente pelo utilizador, o sistema necessita de uma forma de permitir que outros serviços e agentes enviem informações, registos e memórias de volta para a identidade sem que esses serviços necessitem de comunicar em tempo real com o navegador do utilizador. Isto é resolvido através da adoção do padrão *Hybrid Public Key Encryption* (HPKE), uma evolução sofisticada e normalizada de criptografia assimétrica projetada para interoperabilidade global e preparação para a era pós-quântica.

O HPKE é suportado de forma nativa por ambientes V8, incluindo a Cloudflare Workers e browsers modernos, e compõe-se de um Mecanismo de Encapsulamento de Chaves (KEM), uma Função de Derivação de Chaves (KDF) e um algoritmo de Encriptação Autenticada (AEAD). Durante a instanciação da identidade na Cloudflare, o "Twin-User" (o *Durable Object* central) gera um par de chaves assimétricas dedicadas ao agente, utilizando curvas elípticas de alto desempenho como X25519. A chave pública é publicada no perfil da identidade, enquanto a chave privada permanece hermeticamente selada no ambiente de execução do Worker do utilizador, protegida pelo sistema *Cloudflare Secrets Store*.

Quando um sistema externo, ou outro agente no ecossistema, necessita de armazenar um registo sensível ou espelhar dados para os buckets associados a este utilizador, utiliza a chave pública HPKE do utilizador. O sistema externo invoca a rotina de encriptação que gera dinamicamente uma chave simétrica efémera, encripta o conteúdo e encapsula essa chave efémera com a chave pública do agente. O pacote resultante (o texto cifrado e o segredo encapsulado) é entregue à infraestrutura da Cloudflare. O agente do utilizador, detentor da chave privada, pode desencapsular o segredo e processar os dados em segurança. Isto garante que, mesmo que a Cloudflare enquanto transportador inspecione o tráfego, apenas o agente isolado do utilizador consiga processar a informação.

4. O Modelo de Atores e os Durable Objects como "Twin-Users"

O substrato computacional que permite concretizar a premissa de um agente estritamente escopado a uma identidade é baseado na abstração do Modelo de Atores (Actor Model), que a Cloudflare materializa através do serviço *Durable Objects*. O Modelo de Atores, originário de arquiteturas resilientes de telecomunicações, postula que o estado não deve ser partilhado de

forma transversal numa base de dados massiva sujeita a bloqueios (locks); pelo contrário, o estado deve ser encapsulado dentro de atores autónomos que processam mensagens de forma isolada.

4.1. Isolamento Computacional e Consistência Intrínseca

Nesta arquitetura, em vez de implementar um serviço global que gere as memórias e chaves de todos os utilizadores através de consultas complexas a tabelas partilhadas, implementa-se o utilizador propriamente dito. O conceito de "Twin-User" traduz-se na instanciação de um *Durable Object* único e exclusivamente dedicado àquela identidade, invocável globalmente através de um identificador determinístico.

O *Durable Object* assegura consistência forte porque implementa a concorrência através de uma única *thread* de execução. Quando pedidos concorrentes chegam ao sistema (por exemplo, uma atualização de memória proveniente de um serviço externo simultaneamente com uma interação em tempo real do utilizador), o agente processa essas invocações sequencialmente, garantindo que as invariantes de dados da identidade nunca são corrompidas por condições de corrida (race conditions). Este modelo elimina a sobrecarga tradicional de orquestração distribuída, permitindo que a inteligência artificial do agente possa ler e escrever no seu estado sem medo de latência de sincronização.

4.2. Execução Durável, Agendamento e Hibernação

Para que a identidade atue como um verdadeiro agente com "capacidade agêntica", ela não pode ser efêmera, limitando-se a responder a pedidos HTTP. Os agentes devem operar ativamente no contexto dos sistemas em que são inseridos. Através do *Cloudflare Agents SDK*, construído sobre os *Durable Objects*, o "Twin-User" possui acesso a ciclos de vida avançados que superam as limitações da computação *serverless* tradicional.

A capacidade agêntica é ampliada pelo mecanismo de *Fibers* e rotinas de agendamento. Se o agente necessitar de orquestrar chamadas de rede longas, monitorizar eventos externos de terceiros ou executar rotinas de sincronização complexas, o sistema utiliza a função `runFiber()` para isolar esse trabalho. Esta execução durável permite que o agente sobreviva a reimplantações de código, atualizações da plataforma e limites rígidos de tempo de CPU da Cloudflare. O trabalho, medido em minutos ou horas, é mantido vivo de forma transparente. Adicionalmente, a API de *Alarms* e o método `scheduleEvery` permitem que o agente acorde proativamente em horários definidos para executar análises autónomas, sondar as APIs dos serviços onde o utilizador se encontra logado, e tomar decisões automatizadas sem intervenção humana.

Um aspeto económico e de eficiência vital desta abordagem é o mecanismo de hibernação. Quando não ocorrem eventos ativos (nem chamadas web, nem alarmes pendentes), o *Durable Object* serializa o seu estado em disco de forma transparente e entra em modo de hibernação. Neste estado, o consumo computacional cai para zero, e o custo associado à manutenção de milhões de agentes inativos na rede da Cloudflare é anulado. No momento em que um novo evento atinge o sistema, a infraestrutura da Cloudflare redesperta a micro-máquina virtual em poucos milissegundos, restaurando instantaneamente as ligações WebSocket ativas e o contexto do agente.

5. Arquitetura de Memória: Espelhamento de Dados e

Recuperação Semântica

A utilidade de um sistema de identidade baseia-se na sua capacidade de reter e recuperar dados, logs de navegação e metadados contextuais das várias plataformas com as quais interage. O agente do utilizador deve gerir uma malha de dados diversificada, abrangendo desde memórias relacionais rápidas até ao armazenamento em massa de objetos encriptados, mantendo sempre o estrito isolamento inerente ao conceito do *Durable Object*.

5.1. Armazenamento Relacional e FTS5 Local (SQLite)

O coração da retenção de memória da identidade reside na base de dados SQLite incorporada nativamente em cada *Durable Object*. Ao contrário dos motores de armazenamento chave-valor (KV) que requerem serialização complexa, a infraestrutura SQLite-backed dos DOs fornece armazenamento transacional ACID com latência zero (dado que reside na mesma camada física que o código de processamento do agente).

A inteligência própria da identidade constrói o seu contexto recorrendo ao módulo FTS5 (Full-Text Search) ativado nesta base de dados SQLite. Quando o utilizador transita entre plataformas, os eventos (por exemplo, "ação realizada no sistema financeiro X", "preferências configuradas no sistema de saúde Y") são registados em formato JSON estruturado. O agente indexa estes registos no FTS5. Se o módulo de Inteligência Artificial requerer contexto para tomar uma decisão (RAG local - Retrieval-Augmented Generation), o agente executa uma pesquisa semântica ultrarrápida contra a sua base de dados SQLite, reunindo os registos biográficos pertinentes e injetando-os no *prompt* da IA, sem incorrer na latência e no custo de uma chamada externa a um banco de dados vetorial centralizado. A Cloudflare expandiu as capacidades analíticas permitindo o uso de funções matemáticas avançadas sobre as estruturas em SQLite, abrindo caminho para o cálculo de proximidade e avaliação de similaridade de embeddings locais diretamente no ambiente do agente.

5.2. Espelhamento de Buckets Dinâmico e Espaços KV Isolados

Enquanto a base de dados SQLite gere os metadados contextuais, os logs rápidos e as configurações da inteligência artificial, uma identidade agêntica exigirá, ao longo do seu ciclo de vida, o armazenamento de ficheiros estáticos pesados (documentos, configurações estruturais volumosas e anexos). A especificação define que o sistema "salvaria espelhos de buckets/db quaisquer que viesse a ser atrelado ao usuário algum dia".

A infraestrutura atinge este objetivo recorrendo à manipulação sofisticada das ligações de recursos (Bindings) e ao R2 (Cloudflare Object Storage). Através da API do serviço *Workers for Platforms*, sempre que a entidade do utilizador for associada a um novo contexto de armazenamento, um namespace *Workers KV* ou um bucket R2 logicamente isolado é gerado programaticamente. O sistema atualiza os metadados do processo do *Worker* que acolhe a identidade, injetando uma nova variável de ambiente (por exemplo, `USER_DATA_BUCKET`). O código do agente acede a esta variável de forma transparente e isolada (via `env.USER_DATA_BUCKET.put()`). A beleza deste isolamento é que, mesmo que exista código comum entre vários agentes, o ambiente de execução de um utilizador específico nunca terá acesso ao identificador físico ou às referências lógicas do bucket de outro utilizador. Todos os documentos submetidos ao bucket são previamente sujeitos ao ciclo de encriptação WebCrypto AES-GCM delineado anteriormente, garantindo que o R2 e o KV operam apenas como

repositórios "cegos" de *ciphertext*, suportando cenários extremos de privacidade onde a Cloudflare atua meramente como uma rede de distribuição e retenção de blocos de dados sem acesso à informação neles contida.

Tipologia de Memória	Tecnologia Cloudflare	Casos de Uso na Identidade	Isolamento e Segurança
Memória Operacional e Contexto	Embedded SQLite (Durable Objects)	Logs rápidos, sessões ativas, indexação semântica local com FTS5 e metadados contextuais de IA.	Isolado por ID único de DO, sem acesso externo transversal; dados em repouso encriptados nativamente.
Armazenamento de Longo Prazo e Ficheiros	Cloudflare R2 / Workers KV via Bindings	Espelhamento de buckets, documentos estáticos, configurações pesadas retidas permanentemente.	Ligação injetada via metadados de execução (<i>Worker for Platforms</i>). Acessível apenas ao <i>Worker</i> do utilizador. Encriptação <i>Client-Side</i> obrigatória.
Memória Vetorial Densa (Opcional/Avançado)	Cloudflare Vectorize	Suporte a geração aumentada de recuperação semântica (RAG) em larga escala de embeddings.	Roteado pela identidade do agente e particionado logicamente; pesquisa híbrida gerida pelo Workers AI.

6. Integração Modular da Inteligência: Agência, MCP e ACP

Para cumprir o requisito de dotar a identidade com uma "inteligência própria e escopada por usuário processada por IA", garantindo capacidade agêntica para orquestrar chamadas web e efetuar conexões a outros serviços de forma autónoma, a arquitetura abraça dois protocolos cruciais e padronizados pela indústria, garantindo que o agente funcione como a ponte ideal entre a inferência base e a ação executável.

6.1. Model Context Protocol (MCP): O Cérebro em Contacto com a Plataforma

O *Model Context Protocol* (MCP) é um padrão aberto que define o método universal através do qual as aplicações de IA, independentemente do fornecedor do modelo de linguagem (seja o Cloudflare Workers AI com Llama-3, seja uma integração Cloudflare AI Gateway com a OpenAI ou Anthropic), se conectam com o seu ambiente para descobrir, compreender e executar ações sobre recursos externos, funcionando essencialmente como a ligação sensorial de uma inteligência abstrata à base material do sistema e às ferramentas necessárias.

Neste ecossistema focado na identidade, cada "Twin-User" suportado pelo *Durable Object* atua não só como o guardião dos dados, mas também como um Servidor Remoto MCP completo. Utilizando a classe *McpAgent* do *Cloudflare Agents SDK*, a infraestrutura do utilizador define esquemas rigorosamente tipados em TypeScript para todas as ações que a IA pode invocar em

nome da identidade (por exemplo: `read_navigation_logs`, `execute_payment`, `mirror_data_bucket`).

- **Delegações Controladas (Tools) e Code Mode:** O servidor MCP fornece uma listagem explícita dos métodos do utilizador e as respetivas estruturas de dados. A Cloudflare optimizou ainda mais este processo introduzindo a arquitetura *Code Mode*. Em vez de enviar centenas de esquemas de ferramentas que poderiam esgotar a janela de contexto de centenas de milhares de tokens, o servidor MCP do utilizador pode expor apenas a capacidade de execução de código numa *sandbox* (Isolates). O modelo de linguagem escreve autonomamente um *script* TypeScript que o *Worker* executa com rigoroso confinamento, garantindo escalabilidade, uma expressividade imensamente superior na interação com APIs REST, e redução extrema de custos operacionais com consumos massivos de tokens.
- **Autorização OIDC Integrada no MCP:** As chamadas submetidas pelo modelo de linguagem ou pela interface conversacional para o servidor MCP são sistematicamente sujeitas ao rigor do fluxo OAuth 2.1. Como o agente em si engloba o `workers-oauth-provider`, a inteligência apenas consegue operar ferramentas caso possua os privilégios exatos emitidos durante o contexto da sessão ativa, garantindo proteção contra anomalias ou exfiltrações promovidas por eventuais *hallucinations* (alucinações) dos modelos.

6.2. Agent Communication Protocol (ACP): A Malha de Interoperabilidade Multi-Sistema

Enquanto o MCP trata da forma como o raciocínio da IA acede às ferramentas do seu próprio ecossistema isolado, um verdadeiro sistema modular onde a identidade pode "transitar em diversos sistemas" necessita de um protocolo para que os agentes operem entre plataformas, serviços de terceiros e outras identidades de modo coeso e padronizado. A solução para esta comunicação *Agent-to-Agent* (A2A) e *Agent-to-Platform* é a implementação do *Agent Communication Protocol* (ACP).

O ACP é um padrão emergente fundamental que estrutura as comunicações assíncronas através de normas RESTful familiares (ao contrário das infraestruturas puramente dependentes de JSON-RPC utilizadas exclusivamente pelo MCP). Num caso prático, quando a identidade do utilizador (o seu "Twin-User" agente) pretender interagir com a plataforma logística ou analítica de um ecossistema parceiro onde foi efetuado o *login*:

1. **Orquestração A2A Segura:** O agente do utilizador envia uma mensagem padronizada multimodais ACP declarando a sua intenção. A negociação semântica permite que os agentes desconstruam lógicas de negócios complexas sem partilharem bases de código subjacentes nem exporem os dados vitais da infraestrutura Cloudflare do utilizador.
2. **Arquitetura Zero-Trust e Escopo Contextual:** A interação inter-sistemas via ACP é protegida por políticas temporais e de isolamento de contexto (Context-Scoped Anomaly Model). A infraestrutura impede cruzamentos anómalos: se a identidade iniciar dezenas de transações de alto risco devido a um padrão comportamental desviante noutra SaaS, os protocolos de segurança comportamental do ACP (assentes em atestados criptográficos e assinaturas institucionais) suspendem automaticamente a transação sem recorrer a análises genéricas cegas.
3. **Fluxos Longos Assíncronos:** O ACP é desenhado de forma "async-first" e "offline discovery". Esta propriedade harmoniza-se de forma perfeita com o mecanismo de

hibernação dos *Durable Objects*. O agente da identidade submete um pedido prolongado a um sistema parceiro e entra instantaneamente em modo de repouso, preservando os custos e a memória; quando o sistema parceiro responder ao *endpoint* ACP exposto na Cloudflare, o agente regressa ao estado ativo, retoma a execução sem perdas, e armazena os resultados encriptados na sua base SQLite, cumprindo na íntegra os requisitos da "melhor proposta possível" descritos no desenho técnico global.

7. Arquitetura de Isolamento (Multi-Tenancy) via Workers for Platforms

Para garantir formalmente a divisão clara onde "cada usuário = agente = worker central garante que esse deploy por usuário seja bem dividido desde o deploy", evitando interferências laterais (lateral movement) e mantendo as restrições rígidas da encriptação *Zero-Knowledge*, o sistema deve ser construído na fundação do serviço *Workers for Platforms* (WfP) da Cloudflare. O WfP foi projetado exatamente para acomodar e orquestrar de forma confiável código e funções de milhares de instâncias isoladas, mantendo métricas de latência marginais sobre a gigantesca rede de servidores da empresa.

A arquitetura de *Workers for Platforms* instaura três camadas fundamentais de segurança e governança para acomodar a malha dos "Twin-Users":

1. **O Worker de Despacho (Dynamic Dispatch Worker):** Este atua como a interface perimetral única de todos os sistemas de terceiros ou dispositivos. Quando chega um pedido, o Dispatcher inspeciona os cabeçalhos de autenticação, o token OIDC, e a identificação do domínio. O Dispatcher determina matematicamente qual o namespace e qual o *Durable Object* associado a esta identidade exata, encaminhando a solicitação sem qualquer reestruturação complexa de tabelas lógicas, agindo apenas como o controlador de tráfego aéreo.
2. **Namespace Não Confiável (Untrusted Mode):** O sistema deve ser taxativamente configurado e provisionado para operar no modo "Untrusted" (o valor padrão e mais seguro). Ao submeter o *script* do agente do utilizador ao *Dispatch Namespace* neste modo, garante-se que cada invocação executa num ambiente estéril:
 - O agente do utilizador perde a capacidade de inspecionar os objetos *request.cf* partilhados que detêm informações do plano de controlo.
 - A API de Cache global da Cloudflare (*caches.default*) é instantaneamente seccionada; não há qualquer possibilidade de os dados transacionais em memória do Agente A contaminarem ou vazarem acidentalmente para o Agente B.
3. **Controlo de Egressos via Outbound Worker:** O agente de IA poderá usar ferramentas de MCP para contactar "outros serviços" na web. Contudo, permitir o acesso indiscriminado a canais web a uma entidade baseada em modelos de linguagem apresenta riscos graves, como ataques *Server-Side Request Forgery* (SSRF) ou exfiltração de texto plano. Um *Outbound Worker* acoplado ao perfil do utilizador inspeciona e restringe todo e qualquer tráfego emitido a partir do *Twin-User* para o exterior, aplicando sanitização e regras estritas, de modo a garantir que o agente comunica em ACP com plataformas externas aprovadas, sem nunca enviar blocos criptográficos essenciais para a internet pública.

8. Diretrizes de Implementação Orientadas a LLM e

Seleção de Linguagem

A concretização de uma malha tão densa e inovadora requer rigor na escolha dos blocos estruturais de engenharia, tanto no que concerne as linguagens de programação, como nos processos de *deploy* suportados pelos modelos de geração automatizada.

8.1. Linguagem Alinhada com as Melhores Práticas

A Cloudflare Workers é nativamente otimizada para ecossistemas assentes em JavaScript e WebAssembly, devido ao uso extensivo dos *V8 Isolates* do Google Chrome, um modelo que repudia contentores pesados em favor de instâncias executáveis minúsculas. Para arquitetar este sistema, a linguagem preferencial indiscutível é o **TypeScript**, ou em segunda ordem de adoção pragmática, o JavaScript suportado pelas normas ECMAScript recentes (ES2022 ou superior).

O TypeScript não apenas capitaliza sobre a compatibilidade nativa absoluta da Cloudflare (eliminando traduções complexas ou limitações de interface), como dota as definições MCP (que frequentemente assentam na biblioteca Zod para esquematização tipada) de segurança rígida. O TypeScript reduz catastroficamente a probabilidade de erros nas invocações recursivas do FTS5 local via `sql.exec` ou nas configurações dos esquemas criptográficos baseados na Web Crypto API (onde detalhes milimétricos na passagem de metadados Buffer definem a robustez dos protocolos AES-GCM e HPKE). Caso o desenvolvedor opte pelo Rust (ou Go via interoperabilidade avançada), poderá obter ganhos de performance marginais, mas sofrerá grandes impedimentos na orquestração dos componentes centrais de RPC da própria malha da Cloudflare.

8.2. Diagramas de Topologia e Integração do Identificador

A estrutura de como o *pipeline* gerido se desenrola aquando do instanciamento da identidade do utilizador obedece ao seguinte comportamento visual representativo, documentando as transições de *deploy* para *Twin-User*.

```
sequenceDiagram
    participant Plataforma_Terceiros as Plataforma SaaS
    participant User_Browser as Dispositivo (WebAuthn / WebCrypto)
    participant CF_Dispatcher as WfP Dispatcher (Identidade)
    participant Agent_DO as Twin-User (McpAgent DO)
    participant Memory_Store as SQLite & R2 / KV

    Plataforma_Terceiros->>User_Browser: Solicita Acesso à Identidade Universal
    User_Browser->>User_Browser: Gera WebAuthn PRF Output (32-bytes)
    User_Browser->>User_Browser: Deriva Chave Mestra e Chaves HPKE
    User_Browser->>CF_Dispatcher: Inicia Registo / Acesso via OIDC PKCE
    CF_Dispatcher->>CF_Dispatcher: Determina Identificador Pairwise Oculto
    CF_Dispatcher->>Agent_DO: Instancia / Desperta Durable Object
    Agent_DO->>Memory_Store: Executa Migração SQLite FTS5 Local
```

```

Agent_DO->>CF_Dispatcher: Emite Token OAuth Escopado
CF_Dispatcher->>Plataforma_Terceiros: Transmite Token (Identidade
Pairwise e Autônoma Ativa)

loop Ação Agêntica
    Plataforma_Terceiros->>Agent_DO: Invoca Intent ACP / Tool MCP
    Agent_DO->>Memory_Store: Pesquisa RAG Híbrida Semântica /
Dados
    Agent_DO->>Agent_DO: Modela Execução via Trabalhador
Fiber/Stash
    Agent_DO-->>Plataforma_Terceiros: Resposta Assíncrona
Streamada
end

```

9. Conclusão da Proposta Arquitetural

A engenharia pormenorizada de um sistema *AI-Native*, focado na construção de identidades soberanas e transitáveis, propõe a substituição dos modelos baseados em perfis centrais e senhas débeis por um ecossistema ativo de agentes descentralizados. Ao mapear exaustivamente o conceito de uma identidade para a topologia infraestrutural de um *Durable Object* na Cloudflare, este design estabelece que as chaves vitais associadas à descriptação e à privacidade do histórico logado residem, intransponivelmente, sobre o controlo do dispositivo local (via WebAuthn PRF e encriptação *Client-Side* AES-256-GCM), transformando a própria cloud numa transportadora eficiente de pacotes inacessíveis e perfeitamente cegos (Zero-Knowledge).

As necessidades de interação do "Twin-User" num cenário fragmentado (acessando vários serviços sem perder o anonimato correlacional) são suprimidas na perfeição através da adoção de normas OpenID Connect focadas no *Pairwise Subject Identifier*, assegurando o ocultamento e a mitigação completa da exfiltração biográfica passiva entre plataformas. Simultaneamente, as capacidades ativas, proativas e responsivas do agente encontram o seu zénite nos protocolos ACP e MCP, que garantem que as intenções do sistema podem interagir com ferramentas isoladas através de inferências processadas, orquestrar diálogos estruturados intra-SaaS e escalar computação assíncrona, desprovidas dos altos encargos financeiros típicos de outras infraestruturas por se beneficiarem do estado letárgico, inativo e de custo marginal zero da hibernação nativa suportada pela rede de *Edge* global. Este paradigma de sistemas de inteligência integrados na identidade reconfigura definitivamente os standards técnicos de confidencialidade, conectividade sistémica e resiliência das integrações inter-plataforma.

Referências citadas

1. Rethinking State at the Edge with Cloudflare Durable Objects - Jamie Lord, <https://lord.technology/2026/01/12/rethinking-state-at-the-edge-with-cloudflare-durable-objects.html>
2. protectedshare download | SourceForge.net, <https://sourceforge.net/projects/protectedshare/>
3. Enhancing User Privacy with OpenID Connect Pairwise Identifiers - ForgeRock Community,

<https://community.forgerock.com/t/enhancing-user-privacy-with-openid-connect-pairwise-identifiers> 4. Model Context Protocol (MCP) · Cloudflare Agents docs, <https://developers.cloudflare.com/agents/model-context-protocol/> 5. OAuth 2.0 & OpenID Connect: The Complete Guide to What the Standards Actually Say, <https://mrutyunjaypatil.medium.com/oauth-2-0-openid-connect-the-complete-guide-to-what-the-standards-actually-say-e92f040a4251> 6. Cloudflare Durable Objects - Stateful Serverless Functions, <https://www.cloudflare.com/products/durable-objects/> 7. What are Durable Objects? - Cloudflare Docs, <https://developers.cloudflare.com/durable-objects/concepts/what-are-durable-objects/> 8. Moltworker Complete Guide 2026: Running Personal AI Agents on Cloudflare Without Hardware - DEV Community, <https://dev.to/sienna/moltworker-complete-guide-2026-running-personal-ai-agents-on-cloudflare-without-hardware-4a99> 9. Security & Encryption - Afrek, <https://www.afrek.app/help/encryption> 10. Passkey Authentication (WebAuthn) | Median.co, <https://docs.median.co/docs/passkey-authentication> 11. Passkey PRFs for end-to-end encryption - Oblique, <https://oblique.security/blog/passkey-prf/> 12. Two-factor authentication - Cloudflare Fundamentals, <https://developers.cloudflare.com/fundamentals/user-profiles/2fa/> 13. Require hard key auth with Cloudflare Access, <https://blog.cloudflare.com/require-hard-key-auth-with-cloudflare-access/> 14. Passkeys & WebAuthn PRF for End-to-End Encryption (2026) - Corbado, <https://www.corbado.com/blog/passkeys-prf-webauthn> 15. IDPFlare - Identity Provider for Cloudflare, <https://idpflare.com/> 16. OpenID Connect (OIDC) on the Microsoft identity platform, <https://learn.microsoft.com/en-us/entra/identity-platform/v2-protocols-oidc> 17. OAuth provider library for Cloudflare Workers - GitHub, <https://github.com/cloudflare/workers-oauth-provider> 18. Build and deploy Remote Model Context Protocol (MCP) servers to Cloudflare, <https://blog.cloudflare.com/remote-model-context-protocol-servers-mcp/> 19. Authorization · Cloudflare Agents docs, <https://developers.cloudflare.com/agents/model-context-protocol/protocol/authorization/> 20. Pairwise subject IDs · Docs - Connect2id, <https://connect2id.com/products/server/docs/guides/pairwise-subject-identifiers> 21. OpenID Connect Ephemeral Subject Identifier 1.0 - Draft 02, https://openid.net/specs/openid-connect-ephemeral-subject-identifier-1_0.html 22. OpenID provider configuration | PingAM - Ping Identity Docs, <https://docs.pingidentity.com/pingam/8.1/am-oidc1/configure-openid-connect-provider.html> 23. Building a Social Platform with Client-Side End-to-End Encryption - DEV Community, <https://dev.to/webdevtodayjason/building-a-social-platform-with-client-side-end-to-end-encryption-36b2> 24. SELF - Simple. Private. Yours., <https://self.app/terms> 25. Web Crypto · Cloudflare Workers docs, <https://developers.cloudflare.com/workers/runtime-apis/web-crypto/> 26. Web Crypto API - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API 27. Password Encrypting Data with Web Crypto - Brady Joslin, <https://bradyjoslin.com/posts/webcrypto-encryption/> 28. Using HPKE to Encrypt Request Payloads - The Cloudflare Blog, <https://blog.cloudflare.com/using-hpke-to-encrypt-request-payloads/> 29. HPKE: Standardizing public-key encryption (finally!) - The Cloudflare Blog, <https://blog.cloudflare.com/hybrid-public-key-encryption/> 30. hpke - npm, <https://www.npmjs.com/package/hpke> 31. GitHub - dajiaji/hpke-js: A Hybrid Public Key Encryption (HPKE) module built on top of Web Cryptography API., <https://github.com/dajiaji/hpke-js> 32. Asymmetric Encryption for Player PII in Multiplayer Games

- Rob Helmer, <https://www.rhelmer.org/blog/stellar-whiskers-encrypted-pii/> 33. Actor Model Coordination - Agentic Design, <https://agentic-design.ai/patterns/workflow-orchestration/actor-model-coordination> 34. Rules of Durable Objects - Cloudflare Docs, <https://developers.cloudflare.com/durable-objects/best-practices/rules-of-durable-objects/> 35. agents-sdk | Agent Skills Library - Awesome MCP Servers, <https://mcpservers.org/agent-skills/cloudflare/agents-sdk> 36. Project Think: building the next generation of AI agents on Cloudflare, <https://blog.cloudflare.com/project-think/> 37. Agents Changelog | Cloudflare Docs, <https://developers.cloudflare.com/changelog/product/agents/> 38. McpAgent · Cloudflare Agents docs, <https://developers.cloudflare.com/agents/model-context-protocol/apis/agent-api/> 39. Multi-Tenant Platform Development - Cloudflare, <https://www.cloudflare.com/solutions/platforms/> 40. Build Stateful AI Agents - Cloudflare, <https://www.cloudflare.com/products/agents/> 41. SQLite-backed Durable Object Storage - Cloudflare Docs, <https://developers.cloudflare.com/durable-objects/api/sqlite-storage-api/> 42. Unlimited On-Demand Graph Databases with Cloudflare Durable Objects - Boris Tane, <https://boristane.com/blog/durable-objects-graph-databases/> 43. Support or guidance for local vector search in Durable Object SQLite for agent memory · Issue #1472 - GitHub, <https://github.com/cloudflare/agents/issues/1472> 44. SQLite as a Vector Database — Yes, Really - DEV Community, <https://dev.to/zoricic/sqlite-as-a-vector-database-yes-really-4cm4> 45. Bindings - Workers for Platforms - Cloudflare Docs, <https://developers.cloudflare.com/cloudflare-for-platforms/workers-for-platforms/configuration/bindings/> 46. GitHub - jeerovan/secure_file_vault: Client side encrypted cloud storage with BYOK support., https://github.com/jeerovan/secure_file_vault 47. Workers for Platforms - Programmable Solutions - Cloudflare, <https://www.cloudflare.com/products/workers-for-platforms/> 48. How Workers for Platforms works - Cloudflare Docs, <https://developers.cloudflare.com/cloudflare-for-platforms/workers-for-platforms/how-workers-for-platforms-works/> 49. Scaling MCP adoption: Our reference architecture for simpler, safer and cheaper enterprise deployments of MCP - The Cloudflare Blog, <https://blog.cloudflare.com/enterprise-mcp/> 50. Community Providers: Cloudflare AI Gateway - AI SDK, <https://ai-sdk.dev/providers/community-providers/cloudflare-ai-gateway> 51. MCP - Agents - Cloudflare Docs, <https://developers.cloudflare.com/agents/tools/mcp/> 52. Cloudflare's own MCP servers · Cloudflare Agents docs, <https://developers.cloudflare.com/agents/model-context-protocol/cloudflare/servers-for-cloudflare/> 53. What are agents? - Cloudflare Docs, <https://developers.cloudflare.com/agents/concepts/what-are-agents/> 54. Agent Communication Protocol: Welcome, <https://agentcommunicationprotocol.dev/introduction/welcome> 55. What is Agent Communication Protocol (ACP)? - IBM, <https://www.ibm.com/think/topics/agent-communication-protocol> 56. i-am-bee/acp: Open protocol for communication between AI agents, applications, and humans. - GitHub, <https://github.com/i-am-bee/acp> 57. Beyond Context Sharing: A Unified Agent Communication Protocol (ACP) for Secure, Federated, and Autonomous Agent-to-Agent (A2A) Orchestration - arXiv, <https://arxiv.org/html/2602.15055v1> 58. Agent Control Protocol ACP v1.30: Admission Control for Agent Actions Technical Specification and Reference Implementation - arXiv, <https://arxiv.org/html/2603.18829v10> 59. Agent Control Protocol ACP v1.23: Admission Control for Agent Actions Technical Specification and Reference Implementation - arXiv, <https://arxiv.org/html/2603.18829v7> 60. Worker Isolation · Cloudflare for Platforms docs,

<https://developers.cloudflare.com/cloudflare-for-platforms/workers-for-platforms/reference/workers-isolation/> 61. Cloudflare Ships Agent Skills for Zero Trust Deployment... - Develeap, <https://www.develeap.com/news/cloudflare-ships-agent-skills-for-zero-trust-deployment-and-0a080282/> 62. Cloudflare Workers - Global Serverless Functions Platform, <https://www.cloudflare.com/products/workers/> 63. agents/AGENTS.md at main · cloudflare/agents - GitHub, <https://github.com/cloudflare/agents/blob/main/AGENTS.md> 64. Building Vectorize, a distributed vector database, on Cloudflare's Developer Platform, <https://blog.cloudflare.com/building-vectorize-a-distributed-vector-database-on-cloudflare-developer-platform/> 65. Rivet vs Cloudflare Durable Objects, <https://rivet.dev/compare/rivet-vs-cloudflare-durable-objects>